TECHNISCHE UNIVERSITÄT ILMENAU

Faculty of Computer Science and Automation

Department of Systems and Software Engineering

Research Project

Design and Implementation of Web based Interface for MDE4CPP

Submitted by

Harshit Chouhan (71622)

Supervisor:

Dr.-Ing. Ralph Maschotta

Contents

1 Introduction

MDE4CPP [1] is an open-source C++ implementation of the Eclipse Modeling Framework (EMF) core concepts [2]. It acts as a runtime engine that can load and execute any model, let it be Ecore, UML, fUML, OCL or PSCS based [3, 4, 5, 6]. The `PluginFramework` and `GenericApi` at its core supports interaction with any plugin at runtime. This project set out to use the capability of MDE4CPP, developing any sytem via models, and offer it on a browser based interface, that lets users browse and manipulate any kind of model instances directly from a web browser, without any heavy setup or touching any C++ code.

1.1 Motivation and Problem Statement

Although the application could've been available on the system, and could be used via terminal, There was no browser based way to explore or modify MDE4CPP models and develop something out of it. The `GenericApi` already provides everything needed at the C++ level, and the gap only was on the interaction side. Building a web interface that actually uses those capabilities was the central motivation for this work.

Specifically, three challenges stood out. First, there was no intuitive visual interface for browsing and editing model instances at runtime through a browser, as existing access paths required either deep C++ expertise or direct HTTP interaction. Second, for researchers unfamiliar with the MDE4CPP build system or C++ API, the framework remained difficult to approach, and we wanted to lower that barrier significantly. Finally, the `PluginAPI` depends on external Boost C++ libraries [7]; without a standardized way to deliver these, building the service layer on a new machine could fail silently, asking for a more automated technique.

These three issues together motivated the construction of a functional reference prototype, one that could serve as a foundation for future web-based modeling work in the MDE4CPP project.

1.2 Task Definition and Scope

The project was organized around three main tasks. First, refinement of the build infrastructure for the interaction components was done, focusing on machine independent dependency management within the Gradle and CMake pipelines. Second, the C++ `PluginAPI` was explored to understand how best to use its reflective capabilities in a web application. That involved making the native layer serve as the source of truth for both metamodel descriptors and model instances.

The third and most central task was the design and implementation of the web modeler prototype itself. This required a three tier system architecture consisting of a vanilla JavaScript single page application that generates its UI components dynamically from the Ecore metadata it receives, a Node.js middleware layer [8] that sits between the browser and the C++ `PluginAPI` to handle request proxying and error normalization, and the C++ `PluginAPI` itself, which serves as the reflective layer for hierarchical tree retrieval and dynamic instance creation. In the chapters that follow, we document the development of this reference prototype. We start with the MDE4CPP fundamentals, then move to the design rationale, the Ecore utilization, the full technical implementation, and finally an evaluation of the results.

The Ecore meta metamodel [2] is central to understanding what the interface interacts with. Every element visible in the browser such as classifier names, attribute fields, containment trees etc. ultimately maps back to one of the core Ecore meta types. Table 1 lists these types and their role in the interaction layer.

Table 1: Core Ecore meta types and their representation in the interaction layer.

`Meta type`	**What it represents**
`EPackage`	A namespace grouping related classes
`EClass`	A class in the metamodel, has attributes, references, and operations
`EAttribute`	A primitive typed property (e.g. `name: String`, `age: Integer`)
`EReference`	An object typed property; may be a *containment* reference, where the owner controls the child's lifecycle
`EOperation`	$A behavioural feature, a method callable on an instance$
`EObject`	The runtime base type for all model instances

1.3 Project Goals

Beyond the immediate prototype, this work examines how the reflective capabilities of the MDE4CPP `PluginAPI` can be exposed through a three tier web architecture. It looks at what patterns work well when connecting a native C++ modeling runtime to a browser, particularly around proxying, concurrency, and error handling. The project architecture maps directly onto the existing MDE4CPP stack. Table 2 summarises how the core layers relate to the new interaction prototype.

Table 2: Architecture of the MDE4CPP ecosystem and the new prototype.

Layer	Content	Role in Prototype
Generation Pipeline	Ecore/UML input models, Eclipse JAR generators, Gradle generation tasks, CMake build	Source of model structure
Runtime & PluginFramework	`pluginFramework` library, plugin discovery, Ecore reflection API	Reflective Engine
PluginAPI & Prototype Clients	`GenericApi` class; Node.js backend; browser frontend	Interaction Prototype

1.4 Scope of the Interaction Prototype

This project was focused on enabling a new category of interaction with the MDE4CPP core.

One area was the build process for the PluginAPI service layer. The PluginAPI depends on Boost C++ libraries for its embedded HTTP engine (Crow) [9]. To make the prototype reliably deployable across different machines, we introduced an automated dependency delivery mechanism so that the required headers are always available locally, without requiring any manual pre-installation steps.

A second area involved adapting the PluginAPI to the concurrency demands of a live web application. Multiple browser requests can arrive simultaneously, so we needed the C++ layer to handle them safely. This meant introducing a metadata tracking structure for model instances and applying synchronization patterns to protect the shared in-memory object store from concurrent modification.

What was built: The primary contribution is the browser based modeler interface and its Node.js backend [10]. Together they form a three tier application with a Data Access Layer, a Controller Layer, and a Presentation Layer, all working directly with the C++ PluginAPI core.

What was enabled: By studying how to use the PluginAPI for web interaction, we established a prototype that supports classifier reflection and instance tree retrieval. The patterns documented here may be useful for anyone building on MDE4CPP in future.

1.5 Design Questions

The project was shaped around three guiding questions:

1. How can the PluginAPI build be made dependency resilient so the service layer compiles correctly on any target machine?
2. How can the existing in-memory object store be used safely by multiple concurrent web requests without introducing data races?
3. What architectural patterns work best for a three tier web modeler (browser, Node.js proxy, C++ PluginAPI) to ensure correct plugin behavior and long term maintainability?

The report proceeds as follows. Section 2 provides the MDE background, while Section 3 covers the project methodology. Section **??** explores the design rationale and Section 5 provides a deep dive into Ecore utilization. Section 6 describes the technical implementation across all layers. Finally, Section 7 evaluates the results and Section 8 concludes the work.

2 Fundamentals

To understand what this project builds, it would be better to first understand the basic of MDE4CPP it builds on. This chapter covers the core MDE concepts and the specific architecture of MDE4CPP, including how the Ecore meta metamodel works and how the framework moves from a model definition to a running binary.

2.1 Concepts of Model-Driven Engineering

In Model-Driven Engineering [11], models are not just documentation. They are the primary software artifact: executable or transformable representations of a system. Central to this is the concept of metamodeling, where a metamodel defines the rules and structure of a modeling language.

Consider a simple library system. A metamodel for it would say that a "Library" can contain "Books", and that each "Book" has a "Title". That metamodel is itself described using a higher level language. In MDE4CPP, this three level structure consists of the Meta Metamodel, the Metamodel, and the Model Instances. Ecore serves as the Meta Metamodel, defining the primitive building blocks such as classifiers, attributes, and references from which all other models are constructed. The Metamodel then defines the concepts and rules for a specific domain, such as a Library model or the full UML standard. Finally, the Model Instances are the actual runtime objects that conform to a metamodel, representing the entities that are created, read, updated, and deleted during system execution.

This project provides a way to interact with both the metamodel layer and the instance layer simultaneously, through a unified browser interface.

MDE4CPP uses Ecore as its meta metamodel, in alignment with the Eclipse Modeling Framework (EMF) [2]. But the scope goes well beyond simple Ecore defined models. The framework supports UML [3], fUML (Foundational UML) [4], and PSCS (Precise Semantics of UML Composite Structures) [6] all of which are complex, standardized modeling languages with their own execution semantics. In our prototype, the Ecore meta types are the common vocabulary that lets the interface interact with any of these languages through the same generic reflective hooks. Table 3 shows the key Ecore meta types that appear throughout the prototype.

Table 3: Core Ecore meta types utilized in the MDE4CPP ecosystem.

Meta type	**Conceptual Representation**
`EPackage`	A container for related classifiers, providing namespace management in the generated C++ code.
`EClass`	The blueprint for a model entity. It groups structural features like attributes, references, and operations.
`EAttribute`	A primitive valued property, strings, integers, booleans.
`EReference`	An association between classes. Containment references imply a parent child ownership; non-containment references are cross references.
`EOperation`	A behavioral signature that can be invoked on a model instance.
`EObject`	The universal base type for all runtime instances. It provides the reflective hooks used by the Generic API.

2.2 The MDE4CPP Working Pipeline

The MDE4CPP pipeline converts a model definition into a running native binary through a sequence of well defined steps.

It begins with a domain metamodel a `.ecore` file. The `ecore4CPP.jar` generator reads this file and performs a model to text transformation, producing C++ source code for every `EClass` in the model. The output includes Package and Factory classes (entry points and factory methods for creating instances), Implementation classes (the actual data holders for each domain concept), and Reflection layer code (meta object descriptors that allow the runtime to introspect its own structure).

The generated C++ source is then compiled using a two stage build system. Gradle handles the high level orchestration and dependency management, and CMake generates the platform specific instructions for the C++ compiler, MinGW on Windows, GCC or Clang on Linux. The result is a set of shared libraries: `.dll` files on Windows, `.so` files on Linux. Each library is a self contained plugin that carries both the metamodel structure and the logic needed to manipulate instances. Because models are packaged as dynamic libraries, the core framework remains stable and adding a new model type requires no recompilation of existing code.

At runtime, the `PluginFramework` acts as a central registry and service locator. It dynamically loads compiled plugin libraries on demand. When a plugin loads, it registers its factory and meta object descriptors with the `PluginFramework`. That registration is what allows the framework to handle a model it has never seen before at compile time: each plugin's capabilities are discovered through the plugin interface at load time, not embedded ahead of time.

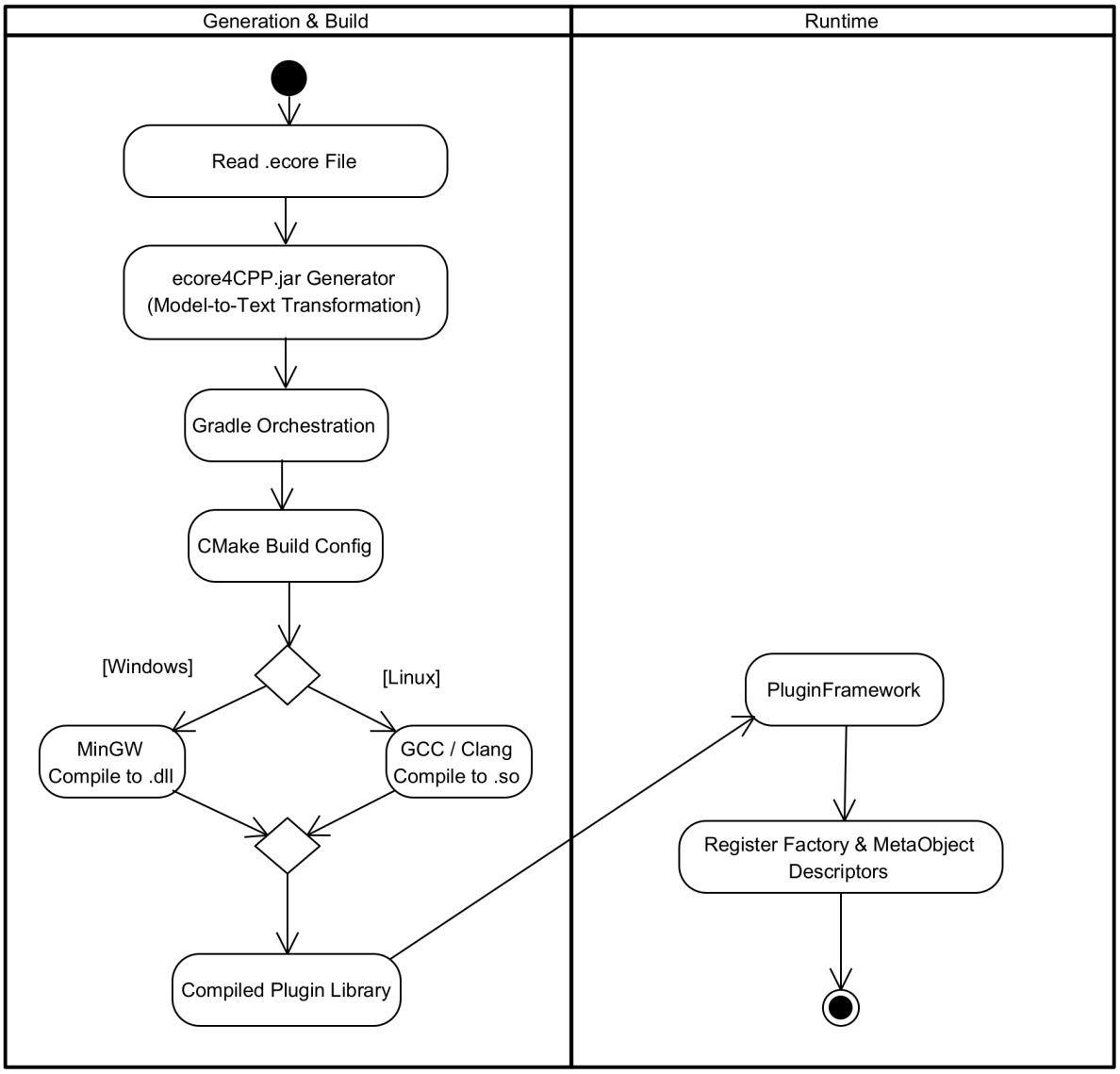


Figure 1: The refined MDE4CPP architecture from metamodel to reflective API exposure.

2.3 Generic API Fundamentals

There are two ways to interact with model instances in MDE4CPP, and the choice between them has a direct impact on how a user interface can be built.

The Specific API is the generated, type safe C++ interface. If you have a `Person` class in your metamodel, the generated code gives you methods like `person->getName()`. This is efficient and safe. But it is inherently tied to a specific model. Building a UI on the Specific API would mean writing new UI code for every distinct model which defeats the purpose of a generic interface.

The Generic API is the approach we use. Every model instance in MDE4CPP inherits from the `EObject` base type, which provides reflective hooks: `eGet(featureName)`, `eSet(featureName, value)`, and `eInvoke(operationName)`. These methods let you

read and write any feature of any object using string based feature names, without needing to know the concrete type at compile time. The prototype can ask the runtime "what attributes does this class have?", receive a list, and render the appropriate input fields. It works the same way for a Library model as it does for a UML model, without any changes to the frontend code.

The Ecore meta types and the reflective MDE4CPP runtime together form the foundation on which the interaction prototype rests. The Generic API is the mechanism that keeps the interface applicable across different plugins and the following chapters describe how we built on top of it.

3 Methodology

The prototype was developed in four phases. Each phase had a specific focus, and together they trace a path from getting the build to work reliably, to discovering how the `PluginAPI` behaves under web conditions, to constructing the interface itself, and finally to validating it.

3.1 Phase 1: Environment Orchestration

The first priority was making sure the prototype could be built and run consistently across different machines. The `PluginAPI` depends on the Boost C++ libraries [7], and without those headers in a predictable location, the build fails. We identified this dependency chain early and developed a Gradle based task to automate header delivery. This gave the rest of the project a stable foundation to work from.

3.2 Phase 2: Reflective Mechanism Discovery

The second phase was a study of the existing `PluginAPI`. We examined which reflective hooks were available and how best to use them in a web facing, multi threaded context. A web server handles concurrent requests, and the MDE4CPP object store was not originally designed with that in mind. This phase produced a working understanding of what the `PluginFramework` and `GenericApi` provide, and what additions were needed to make them safe for simultaneous access by multiple web requests.

3.3 Phase 3: Prototype Interface Construction

The third phase was the main construction effort. The vanilla JavaScript frontend, the Node.js middleware proxy [8], and the C++ reflective layer were developed together in an iterative cycle. The guiding principle was metamodel neutrality: at no point should the frontend assume it knows what model is loaded. It always queries the `PluginAPI` for the current structure and builds its interface from the response.

3.4 Phase 4: Multi Metamodel Validation

The final phase tested the prototype against several different domain models to verify its generic adaptability. We observed how the interface responded to models with varying structural complexity, simple class models on one end, deep UML containment hierarchies on the other, and evaluated both the correctness of the visual representation and the consistency of the underlying object state.

4 Concepts

This chapter examines the architecture of the MDE4CPP interaction layer. We focus on why certain decisions were made for the three-tier system and the structure of both the frontend and backend. It appears that two principles guided these choices. First, we aimed to maximize the utility of the MDE4CPP reflection API. Second, we maintained a strict separation of concerns. One might argue that this ensures the prototype stays readable and easy to extend.

4.1 Design Rationale

Every technical choice in this project addressed a specific need. And the resulting structure came from an iterative process. We had to bridge a fast C++ core with a stateless web browser, which wasn't always straightforward. It was observed that bridging these two worlds required a careful approach to data flow.

A three-tier structure is used here. It consists of the Browser, a Node.js middleware, and the C++ reflective core. We chose this split to provide a buffer between the native runtime and the web. The Node.js layer works as a gatekeeper. It handles tasks like CORS headers and request validation. This lets the C++ core focus only on model management. But the middleware does more than that. It catches raw C++ responses and turns them into clean JSON errors. So the frontend gets a predictable format even when things go wrong. In addition, this separation allows for fallback mechanisms. The Node.js layer can serve cached data if the C++ process is down. This prevents the UI from hanging.

The Node.js backend follows a classic layered pattern. It uses routes, controllers, and services. The routes define the API surface without needing to know the processing details. Controllers then extract and validate inputs. This keeps the business logic free from HTTP-specific code. The service layer is the heart of the backend. It coordinates directly with the `PluginAPI`. This separation makes it simple to add features like logging without breaking other parts.

The frontend is split into three layers: Data Access, Controller, and Presentation. Data Access modules talk to the backend. If the API format changes, we only need to update this layer. The Controller layer uses the `ModelerApp` to centralize state. This avoids messy dependencies between panels. Finally, the Presentation layer handles the UI. These components just render the data they receive. They're easier to debug because their behavior is purely visual.

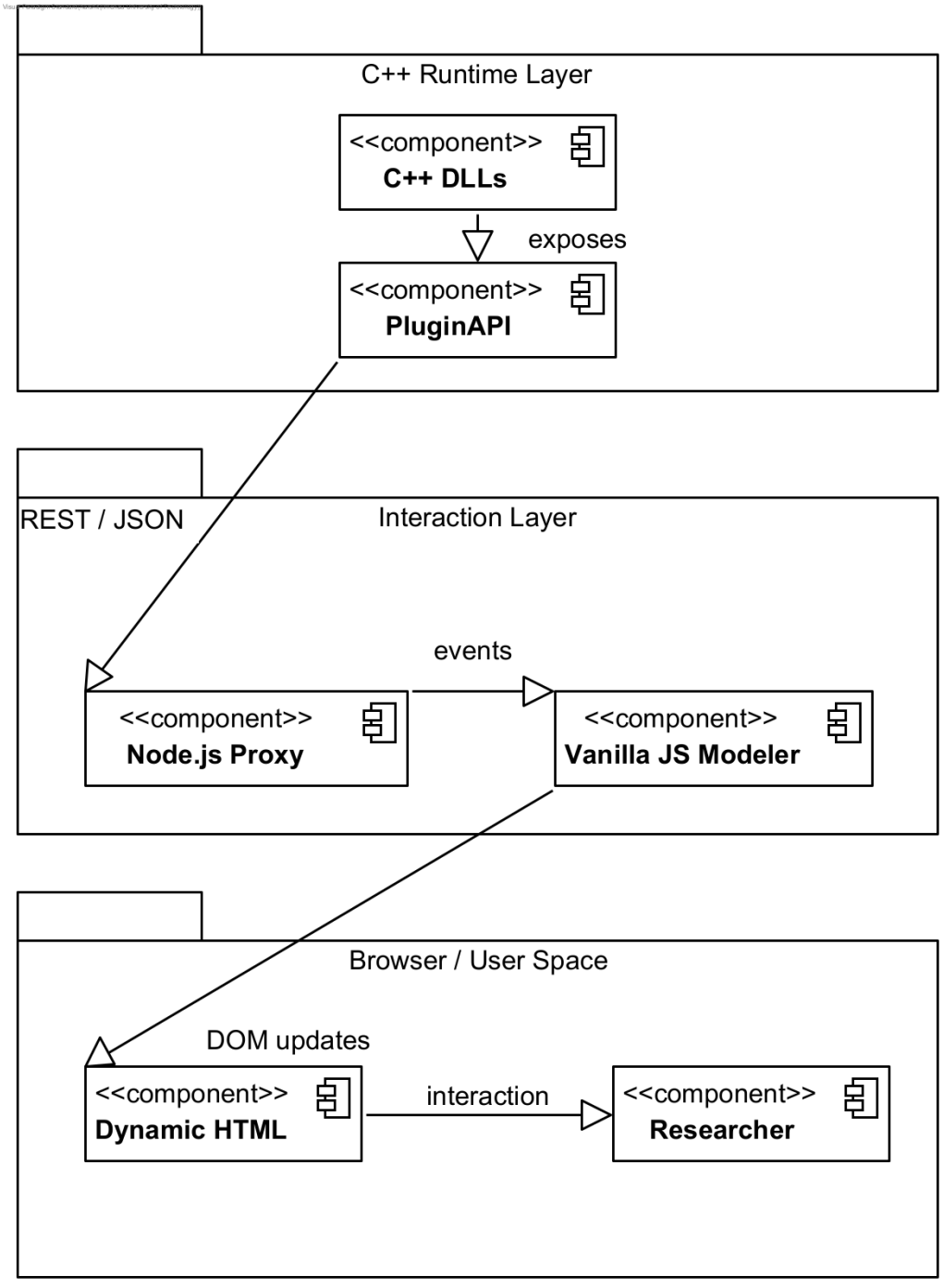


Figure 2: Establishing a Three-Tier Interaction Layer for C++ Model Runtimes.

4.2 Conceptual Interaction Flow

The core interaction pattern is simple. Metadata flows from the runtime to the browser to describe the language. Then user actions flow back down as generic model changes.

4.3 User Interface Layout and Presentation Rationale

Beyond the three-tier architecture, the specific layout and presentation of the modeler were guided by the need for high information density and structural clarity.

4.3.1 Contextual Navigation with Dropdown Menus

We used a dropdown menu in the header for plugin selection. This keeps the focus on the active modeling context. MDE4CPP models can be quite large, so space matters. By using a dropdown, we ensure components like the `UnifiedTree` and `PropertiesPanel` can pivot to a specific metamodel without distraction. It also saves screen real estate. Sidebars are better used for deep model exploration. Since switching plugins is rare, this model is more efficient.

4.3.2 Model Hierarchy Representation

A recursive tree structure was chosen for model navigation. It reflects the natural hierarchy of Ecore models. Containment references define ownership, and a tree is the best way to show how objects belong together. A flat list might obscure these bonds. It would be hard to tell a root element from a nested child. And the tree provides spatial context. Users can see exactly where they are in complex recursive structures.

4.3.3 Property Management via Tabbed Interfaces

The `PropertiesPanel` uses a tabbed interface. We categorized attributes, references, and operations separately. This gives categorical clarity. It prevents information overload because users only see what they need. For example, if someone is editing a name, they shouldn't have to scroll past cross-references. This approach scales well for dense metamodels like UML.

4.3.4 Dynamic Object Creation with Contextual Palettes

The `CreationPalette` is a floating, contextual element. It isn't a fixed sidebar. We did this to ensure semantic correctness. Not every classifier is a valid child for every instance. The palette only shows types that can be created under the current selection. This prevents users from building invalid models. It also reduces cognitive load.

Table 4: Conceptual overview of UI component responsibilities.

Component	**Mapped Concept**	**Responsibility**
UnifiedTree	Containment Hierarchy	Provides the primary navigation through both the Metamodel (classes) and the Model (instances).
PropertiesPanel	EStructuralFeatures	Reflection-based editor that generates input fields for EAttributes and selection widgets for EReferences.
CreationPalette	EClass Instantiation	Filtered list of valid classifiers that can be created as children of the current selection.
CommandManager	Operational Integrity	Manages the transaction-like execution of actions, enabling undo/redo.
JsonViewer	Serialization	Real-time reflection of the in-memory state in its canonical JSON or XMI format.

4.3.5 Live Data Visualization

We integrated a live JSON and XMI viewer directly into the main panel. This helps bridge the gap between the UI and the underlying data. Researchers get immediate confirmation of their edits. The real-time feedback helps users understand the framework's mapping logic. By seeing the raw state alongside the graphics, users might gain a deeper understanding of the C++ core.

4.3.6 Breadcrumb Navigation

The `MainEditor` uses a breadcrumb trail to help users stay oriented. UML models often have deep nesting, and it's easy to get lost. Breadcrumbs provide a clickable path back to any ancestor. This complements the vertical tree view. It allows for quick backtracking without rescanning the sidebar.

4.3.7 Contextual Action Menus

The `ContextMenu` gives quick access to actions like deletion. It appears right at the point of interaction. This means users don't have to move their mouse to a global toolbar. It also reduces clutter. The menu only shows actions that make sense for the selected element. For instance, you won't see an "invoke operation" action unless it's a model instance.

4.4 Component Overview and Responsibilities

The interface is built from several specialized components. Each one maps to an MDE concept. Table 4 summarizes these roles.

4.5 Plugin-Agnostic Interaction Principle

The interface should work with any MDE4CPP plugin without changes. We achieved this through metamodel sensitivity. The UI has no hardcoded forms. Instead, it reacts to the descriptors it receives at runtime. If a "Library" plugin loads, the UI shows library concepts. This avoids the hardcoding trap. A single binary runtime can support many different domain-specific languages.

Centralized selection management in the `ModelerApp` keeps panels in sync. When a node is selected in the `UnifiedTree`, the event goes to the orchestrator. Then it updates the `PropertiesPanel` and `JsonViewer`. This hub-and-spoke model is cleaner than having components talk to each other directly. It makes state changes more predictable. And it's easier to debug or log.

4.6 Rationale for the Command Pattern

We used the Command Pattern [12] for model changes. This includes things like updating an attribute or creating a link. The pattern lets us encapsulate an action and its inverse. This gives us reliable undo and redo. It also provides an audit trail of changes. This is useful for both documentation and debugging. By separating the logic from the UI buttons, we kept the architecture easier to maintain.

In summary, the architecture connects the C++ runtime to a browser through specific layers. These layers prioritize safety and flexibility. The design provides a functional tool and a roadmap for web-based model interaction.

Table 5: Mapping Ecore concepts to interaction prototype features.

Ecore Concept	Prototype Implementation
EClass Reflection	Used by the *Creation Palette* to identify valid object types for the currently loaded plugin and display them as instantiation options.
EAttribute Metadata	Drives form generation in the *Properties Panel*. The interface interprets the attribute type (EString, EInt, etc.) and renders the appropriate input control with validation.
Containment References	The structural backbone of the *Unified Model Tree*. The interface uses containment flags to distinguish ownership relationships from cross-references.
Reflective Hooks (`eGet`/`eSet`)	The technical bridge inside the C++ layer. The `GenericApi` uses these to read and write model properties without needing to know the concrete compiled type.

5 Discussion

How well the prototype works depends almost entirely on how accurately it reads and uses the underlying Ecore structures. This chapter discusses how specific Ecore concepts were put to use in the interaction layer and what trade-offs appeared along the way.

5.1 Conceptual Mapping to Prototype Features

Every feature in the interface traces back to a specific Ecore concept. Table 5 shows how the core meta-types map to the functional parts of the prototype.

5.2 Utilization of the Generic API Layer

One of the more significant findings during development was how capable the MDE4CPP Generic API actually is. By treating every model object as an `EObject` and going through the reflective hooks, it becomes possible to perform operations on instances of any class Library, UML, fUML, it does not matter without any type-specific code in the interface layer. The `GenericApi` class in the `PluginAPI` translates string based feature identifiers into precise operations on the native instances. And that translation is what makes the whole approach work.

5.3 Hierarchical Integrity and Model Containment

MDE4CPP allows instances to exist as standalone objects in the runtime not every object needs to be contained in a parent. But for the prototype's tree view to make sense, we needed a consistent visual hierarchy. So we decided during the discovery phase that object creation in the tree would require specifying a valid containment reference. This

is a prototype-level decision: the interface enforces a parent-child relationship for tree placement, even though the underlying runtime would not strictly require one. The parent name is tracked in the instance store alongside each object's class and plugin — a lightweight bookkeeping mechanism introduced for the prototype so the tree could be reconstructed from metadata alone, without traversing the Ecore object graph. It keeps the visual model consistent with the EMF containment hierarchy and makes the tree easier to reason about.

5.4 Model Serialization and XMI Compatibility

Model states in the browser are represented as canonical JSON, which is straightforward to build from the `eGet` responses. But the wider MDE ecosystem does not use JSON. To ensure models created via the prototype can be imported back into tools like Eclipse or Papyrus, we implemented XMI 2.0 export [13]. The `xmiSerializer` module converts the canonical JSON representation into a schema-compliant XML document. This makes the prototype a compatible participant in the broader MDE toolchain, not just a standalone viewer.

Together, these patterns show that the interaction layer is not just a display for model data. It is a technically grounded interface that reads and writes according to the formal Ecore specification mediated by the `GenericApi` and made interoperable through XMI.

6 Implementation

This chapter traces the technical implementation of the three-tier interaction layer, following the path a request takes from the C++ PluginAPI upward through the Node.js proxy to the browser-side modeler. Each layer is described in terms of its responsibilities, its internal structure, and how it connects to the layers on either side.

6.1 C++ PluginAPI Enhancement

The PluginAPI resides in `src/common/MDE4CPP_PluginAPI` and provides the reflective interface into MDE4CPP. The work here focused on two areas: making the build reliable across different machines, and introducing thread safety alongside instance tracking to support concurrent web requests.

The C++ service layer depends on the Boost C++ libraries [7] (version 1.82.0) for the Crow HTTP engine [9]. Without those headers, the build will simply fail. To handle this, a custom Gradle task named `deliverBoostLibrary` was introduced in the root `build.gradle` file. This task follows a three-step resolution process: existing Boost headers in `MDE4CPP_HOME` are checked first, the `BOOST_ROOT` environment variable is consulted if the first check finds nothing, and the required version is downloaded automatically from official archives if no local installation can be located.

The CMake configuration for the PluginAPI was also updated to check this local directory before looking at system-wide paths. This gives the build a predictable Boost location regardless of which machine it runs on.

Prototype Interaction Management. MDE4CPP natively supports model navigation through `EObject` references. For this prototype, however, a simpler intermediate mapping was introduced; it associates logical string-based instance names with their native runtime components. This made the asynchronous data flow between the web layer and the C++ runtime easier to reason about during the discovery phase. To protect this shared mapping from concurrent web requests, it was wrapped in a mutex-protected structure, and a targeted synchronization pattern was applied: the handler checks for naming conflicts under lock, releases the lock during the slow JSON parsing step, and re-acquires it only to commit the new instance. This keeps critical sections short while still guaranteeing consistency.

6.1.1 HTTP Routes

Table 6 shows how frontend calls are mapped through Node.js to the C++ PluginAPI.

Table 6: Route mapping across all three layers. Technical identifiers like `:p` represent parameters.

Frontend call	Node backend route	C++ PluginAPI route
`getClassifiers(p)`	`GET/api/v1/plugins/:p/classifiers`	`GET/:p/classifiers`
`getClassifierDetails(p,c)`	`GET/api/v1/plugins/:p/classifiers/:c`	`GET/:p/classifiers/:c`
`getModelTree(p)`	`GET/api/v1/plugins/:p/objects/tree`	`GET/:p/tree`
`createFromClassifier(p,c,n)`	`POST/api/v1/plugins/:p/classifiers/:c/create`	`POST/:p/objects/:c/:n`
`deleteInstance(id)`	`DELETE/api/v1/plugins/objects/:id`	`DELETE/:p/objects/:c/:id`

Plugin discovery. `GET /plugins` reads `m_plugins`, builds a JSON array of plugin names, and returns HTTP 200. No locking is required here because `m_plugins` is populated once at startup and is never changed afterward.

Classifier reflection. `GET /:plugin/classifiers` and `GET /:plugin/classifiers/:name` expose the metamodel structure of a loaded plugin. The detail route returns the full list of attributes, references, and operations for a named classifier, including types, multiplicities, and containment flags.

Instance management. `POST /:plugin/objects/:class/:name` uses the split-phase locking approach described above and returns HTTP 201. `GET /:plugin/objects/:class/:name` acquires the lock, copies the pointer and metadata, releases the lock, and then serializes the result using `writeValue`. The update route calls `readValue` on the incoming JSON delta. Delete acquires the lock and removes the entry.

Instance tree. `GET /:plugin/tree` returns the full instance hierarchy for a plugin, as described in the next subsection.

6.1.2 Endpoint Implementation Details

`GET /plugins` The handler reads the `m_plugins` map, extracts each plugin's name, constructs a JSON array, and returns HTTP 200. Because `m_plugins` never changes after startup, no locking is required.

`POST /:plugin/objects/:class/:name` The instance map is checked under lock first; HTTP 400 is returned if the name already exists. If not, the lock is released, the JSON

request body is parsed, and `readValue` is called to build a new `EObject`. On success, the lock is re-acquired, the entry is inserted, and HTTP 201 is returned.

`GET /:plugin/objects/:class/:name` The mutex is acquired, the requested name is looked up, the stored plugin name is verified against the URL parameter, and then the pointer and metadata are copied before the lock is released. `writeValue` serializes the object, and HTTP 200 is returned.

6.1.3 JSON Ecore Conversion

`readValue` and `writeValue` handle the translation between JSON and Ecore objects. `readValue` takes a plugin name, a classifier name, and a JSON object; it resolves the classifier in the plugin's EPackage, creates a new `EObject` via the plugin factory, and then iterates over the JSON fields to set attribute and reference values with type conversion. `writeValue` does the reverse: given an `EObject`, it reads the runtime type, iterates over the classifier's structural features, and serializes their values into a JSON object.

6.1.4 Building the Instance Tree

`GET /:plugin/tree` reconstructs the instance hierarchy from instance metadata alone, without traversing the Ecore object graph. The algorithm has four steps: (1) the mutex is acquired and all entries for the requested plugin are snapshotted into local maps tracking parent-child relationships and root nodes, then the lock is released; (2) JSON nodes are built recursively for each root; (3) a depth limit is enforced to prevent stack overflow; (4) cycles are detected by tracking visited names on the current recursion path. The C++ implementation of the first two steps is shown below.

```
// Step 1: snapshot metadata under lock, then release
std::map<std::string,
         std::pair<std::string, std::string>> nameToInfo; // name
            -> {class, parent}
std::map<std::string, std::vector<std::string>> parentToChildren;
std::vector<std::string> rootNames;
{
    std::lock_guard<std::mutex> lock(m_objectsMutex);
    for (const auto& [name, so] : m_objects) {
        if (so.pluginName != plugin_name) continue;
        nameToInfo[name] = {so.className, so.parentName};
        if (so.parentName.empty()) rootNames.push_back(name);
        else parentToChildren[so.parentName].push_back(name);
    }
}
```

```
15
16 // Step 2: recursive build with cycle detection
17 std::set<std::string> visited;
18 std::function<crow::json::wvalue(const std::string&, size_t)>
19 buildNode = [&](const std::string& name, size_t depth) -> crow::
       json::wvalue {
20     crow::json::wvalue node;
21     if (depth >= 100 || visited.count(name)) {
22         node["name"] = name; node["type"] = "cycle"; return node;
23     }
24     visited.insert(name);
25     node["name"] = name;
26     node["type"] = nameToInfo[name].first;
27     if (parentToChildren.count(name)) {
28         auto children = crow::json::wvalue::list();
29         int i = 0;
30         for (const auto& child : parentToChildren[name])
31             children[i++] = buildNode(child, depth + 1);
32         node["children"] = std::move(children);
33     }
34     return node;
35 };
```

Listing 1: Snapshot and recursive build in the `GET /:plugin/tree` handler.

A few design points are worth noting here. The mutex is released before the recursive build begins, since the local maps hold only strings and no lock is needed during tree construction. The `visited` set prevents infinite loops where parent metadata contains a cycle. And `node["type"]` is drawn from the stored `className` string; no Ecore reflection calls happen during the tree build. The resulting JSON takes the following shape:

```
{ "roots": [
    { "name": "MainLibrary", "type": "Library",
      "children": [
        { "name": "Book1", "type": "Book", "children": [] }
      ]}
  ]}
```

6.1.5 Errors

The PluginAPI returns specific HTTP codes for each error type. An unknown plugin or classifier produces 404, a duplicate instance name or malformed JSON produces 400, and

internal reflection failures produce 500. This gives the Node backend enough information to return a meaningful error to the browser.

6.1.6 Entry Point

`MDE4CPP_PluginAPI_Main.cpp` is intentionally minimal. It reads `MDE4CPP_HOME`, initializes the `PluginFramework` singleton, calls `GenericApi::eInstance` to trigger route registration, installs signal handlers for graceful shutdown, and enters the main loop while Crow's worker threads handle incoming requests.

6.2 Node.js Backend Proxy

6.2.1 Middleware Architecture

The Node backend sits between the browser and the C++ PluginAPI. From the browser's side, it looks like a stable, versioned JSON API. From the C++ side, it is just an HTTP client. Beyond routing, CORS policy, error normalization, and request validation are handled here, along with a fallback mode that serves in-memory mock data when the C++ process is unavailable. This was implemented using Node.js [8] and the Express framework [10].

Figure 3 shows the request path through the backend.

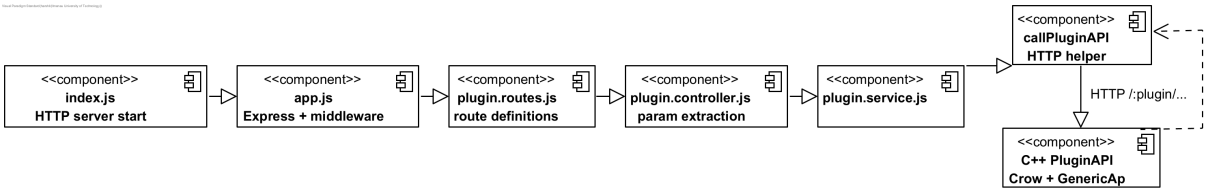


Figure 3: Request path through the Node.js backend.

6.2.2 Server Initialization

`index.js` starts the HTTP server; `app.js` builds and configures the Express app. Keeping these separate means tests can import `app.js` without binding a port. The middleware stack in `app.js` is assembled in a fixed order: CORS first, then body parsers, then the `/api/v1` router, and finally the error handler. The order matters here. CORS headers must be set before any route runs, and the error handler must come last to catch anything the routes throw.

6.2.3 Routes, Controllers, and Service

`plugin.routes.js` declares route registrations, pairing each HTTP method and path with a controller function wrapped in `asyncHandler`. `plugin.controller.js` extracts

parameters from `req.params` and `req.body` and calls the appropriate service method. The actual logic lives in `plugin.service.js`, which defines the `callPluginAPI` helper and implements the higher-level methods with fallback logic. For example, `getAllPlugins` tries `/plugins/details`, then `/plugins`, and then mock data. This fallback keeps the modeler usable even when the C++ service is offline.

6.2.4 Config and Error Handling

`src/config/index.js` exports a frozen configuration object that reads environment variables for port, PluginAPI URL, CORS origins, and MDE4CPP paths. `src/middleware/errorHandler.js` provides two things: `asyncHandler`, which wraps async route handlers so promise rejections are forwarded to the error handler, and `errorHandler` itself, which logs the error and returns a JSON payload with a code, message, optional details, and timestamp.

6.3 Modeler Frontend Interaction Layer

6.3.1 Overview

The modeler frontend is a single-page vanilla JavaScript application organized into three layers. Table 7 maps each module to its layer and main responsibility.

Table 7: Frontend modules, their layers, and responsibilities.

Module	Layer	What it does	Depends on
`plugin-api.js`	Data Access	HTTP calls; URL building; safe JSON parsing	browser `fetch`
`MetamodelService`	Data Access	Classifier queries; containment tree; valid child types	`plugin-api.js`
`ModelService`	Data Access	Instance CRUD; tree fetch; attribute and reference updates	`plugin-api.js`
`ValidationService`	Data Access	Client-side constraint checking	`MetamodelService`
`ModelerApp`	Controller	Global state; event routing; command dispatch	all services + components
`CommandManager`	Controller	Undo/redo stacks	command objects
`UnifiedTree`	Presentation	Metamodel + instance tree; node selection	`MetamodelService`, `ModelService`
`PropertiesPanel`	Presentation	Tabbed attribute/reference editor	`MetamodelService`, `ModelService`
`MainEditor`	Presentation	Breadcrumb trail; local children view	`MetamodelService`, `ModelService`
`CreationPalette`	Presentation	Valid child type list	`MetamodelService`
`ContextMenu`	Presentation	Right-click actions	`ModelerApp` callbacks
`CreateInstanceDialog`	Presentation	Instance creation form and validation	`ValidationService`, `ModelerApp`
`ModelJsonViewer`	Presentation	JSON/XMI view; copy; download	`modelSerializer`, `xmiSerializer`
`modelSerializer`	Utilities	Builds canonical JSON document	`MetamodelService`, `ModelService`
`xmiSerializer`	Utilities	Converts canonical JSON to XMI 2.0	`modelSerializer` output

6.3.2 Frontend Structure

`modeler.html` defines the DOM structure and boots the application. The body contains named regions: a header with the plugin selector and toolbar buttons, a left sidebar (`#unified-tree`), a central panel (`#diagram-panel`), a right panel (`#properties-content`), and a bottom panel (`#children-content`). Floating containers hold the creation palette, context menu, and modal dialogs.

Scripts are loaded in dependency order, utilities first, then services, then UI components, because each module depends on the ones loaded before it. The final inline script waits for `DOMContentLoaded`, creates a `ModelerApp`, and calls `init()`.

6.3.3 Data Management Services

The four data access modules are the only ones that communicate with the backend. No UI component issues HTTP calls directly; all data needs are routed through one of these services. This separation is deliberate: it simplifies testing and makes it possible to replace any individual service without touching the UI.

`plugin-api.js` is the lowest-level module and the only one that issues HTTP requests to the Node backend. Its role is to translate a set of named, intent-oriented function calls into raw HTTP exchanges, returning either parsed JSON or descriptive error objects.

The following public functions are exposed:

- `getPlugins()`
- `getClassifiers(pluginName)`
- `getClassifierDetails(pluginName, className)`
- `getObjectTree(pluginName)`
- `createFromClassifier(pluginName, className, instanceName, properties)`
- `createChildObject(...)`
- `setFeatureValue(...)`
- `deleteInstance(instanceId)`

All of them use the internal helper `safeJsonParse(response)`.

`safeJsonParse` first checks the `Content-Type` header of the response. If the content type is not JSON, as would happen if Node returned an Express error page, a descriptive error is thrown before any attempt is made to parse the body. If the type is JSON but parsing fails, an error is thrown as well. Without this guard, backend error responses would either cause uncaught runtime exceptions or silently propagate `null` values through the service stack.

`plugin-api.js` has no knowledge of any UI component and maintains no state between calls. This narrow interface makes it straightforward to mock in tests.

`MetamodelService` encapsulates all knowledge about metamodel structure, shielding UI components from the raw JSON shapes returned by the backend's classifier responses. It is constructed by `ModelerApp` and maintains an internal cache keyed by plugin name and classifier name. Its public interface consists of four methods:

- `getClassifiers(pluginName)`
- `getClassifierDetails(pluginName, className)`
- `getContainmentTree(pluginName)`
- `getValidChildTypes(pluginName, parentClassName)`

`getClassifierDetails` checks the cache first; on a miss, `plugin-api.getClassifierDetails` is called, the cache is populated, and the result is returned. `getContainmentTree` builds a logical classifier graph by first fetching the classifier list, then fetching details for each, and assembling the containment relationships into a structure that `UnifiedTree` can render directly. `getValidChildTypes` uses containment reference information from classifier details to compute which classifier types may legally be created as children of a given parent, including cardinality filtering.

`ModelService` encapsulates all operations on runtime instances. Its public methods map directly to the actions a user takes in the modeler. A lightweight cache of the most recently fetched instance tree may be maintained to reduce redundant backend calls. Its public interface includes:

- `getModelTree(pluginName)`
- `createRootInstance(...)`
- `createChildInstance(...)`
- `deleteInstance(...)`
- `updateAttribute(...)`
- `setReference(...)`
- `getObjectDetails(...)`
- `getObjectAttributes(...)`

`createChildInstance` differs from `createRootInstance` in that a parent instance name and containment reference identifier are included in the request body, allowing the C++ PluginAPI to establish the correct parent-child relationship in its instance tracking. Both `updateAttribute` and `setReference` call `plugin-api.setFeatureValue`, leaving type encoding to `plugin-api.js`.

`ValidationService` catches constraint violations in the browser before any request is sent. It gives the user immediate feedback without a server round-trip. It is constructed by `ModelerApp` and holds a reference to `MetamodelService` for accessing classifier constraints. Methods include `validateInstanceCreation(...)`, `validateAttributeUpdate(...)`, and `validateCardinality(...)`. When a check fails, a structured result with error messages is returned so the calling component can display them inline. `ValidationService` never calls `plugin-api.js` directly and does not touch the DOM.

All four modules collaborate consistently: UI requests go through `MetamodelService` or `ModelService`, those services call `plugin-api.js`, and `plugin-api.js` issues the actual HTTP request. `ValidationService` runs synchronously before any mutating request is dispatched. This means mocking or replacing any single module requires no changes to the modules above it.

6.3.4 Application Logic and State Management

The Controller Layer has two modules: `ModelerApp.js`, which owns global state and routes events, and `CommandManager.js`, which implements the undo/redo infrastructure.

`ModelerApp` is the single owner of global application state and the coordination point for cross-component communication. It is instantiated once at startup. Its state is minimal by design: `currentPlugin` holds the name of the currently displayed plugin, and `selectedNode` holds the currently selected tree node descriptor. Local state such as which tab is open in `PropertiesPanel` or which branches are expanded in `UnifiedTree` is owned by the respective components, not by `ModelerApp`.

The `init()` function sets up the application in a fixed sequence. DOM discovery comes first, querying and storing required element identifiers, followed by the construction of `MetamodelService`, `ModelService`, and `ValidationService`. Once the services are ready, each UI component is instantiated with a reference to the main `ModelerApp` for coordination. Keyboard shortcuts and DOM event listeners are then wired up, and an initial fetch is performed via `plugin-api.getPlugins()` to populate the plugin selector.

The most important runtime behavior is selection propagation. When a user clicks a tree node, `UnifiedTree` calls `app.onNodeSelect(descriptor)`. `ModelerApp` assigns the descriptor to `selectedNode` and propagates it to all dependent components in sequence:

- `propertiesPanel.showNode(descriptor)`
- `mainEditor.showNode(descriptor)`
- `creationPalette.updateForNode(descriptor)`
- `modelJsonViewer.refresh()`

Centralizing this in one method means all panels consistently reflect the same selection state.

Mutating actions such as instance creation, attribute updates, and deletion are handled by methods like `onCreateInstance`, `onUpdateAttribute`, and `onDeleteInstance`. Each constructs a command object and passes it to `commandManager.executeCommand`. `ModelerApp` never calls `ModelService` directly for mutations; every write goes through the command infrastructure.

`CommandManager` implements the command pattern, providing undo and redo semantics for all model-mutating actions. Two stacks are maintained: `historyStack` for executed commands and `redoStack` for undone commands. Each command object implements `execute()`, which performs the action through `ModelService`, and `undo()`, which performs the inverse using values captured at construction time.

When `executeCommand(cmd)` is called, the manager calls `cmd.execute()`, pushes the

command onto `historyStack`, and clears `redoStack`, since a new action invalidates the previously undone sequence. When `undo()` is called, the top command is popped, `cmd.undo()` is called, and the command is pushed onto `redoStack`. `redo()` is the reverse of this process.

6.3.5 User Interface Components

The Presentation Layer contains all modules responsible for rendering model data and receiving user input. These are described in three groups: the tree and navigation components, the editing and inspection components, and the creation and context menu components.

`UnifiedTree` renders the left-hand panel, presenting a navigable view of both the metamodel containment structure and the runtime instance hierarchy. It is the primary entry point for exploring models. Three internal state variables are maintained: `treeData` (the merged metamodel plus instance structure), `selectedNode` (the currently highlighted node descriptor), and `expandedNodes` (a set of node keys recording which branches are open). Data is drawn from `MetamodelService` and `ModelService`, and callbacks into `ModelerApp` are issued via `onNodeSelect` and `onNodeRightClick`. Model state is not modified by this component.

The internal `treeData` structure takes the following form:

```
{
  plugin: "LibraryModel",
  metamodel: {
    classifiers: [ { name, attributes, references, operations }, ... ]
  },
  instances: {
    roots: [ { name, type, children: [ ... ] }, ... ]
  }
}
```

Each rendered DOM node carries a descriptor encoding the node type (`"plugin"`, `"classifier"`, `"reference"`, or `"instance"`), the plugin name, and where applicable the classifier name and instance identifier. When `loadPlugin(pluginName)` is called, two parallel requests are fired: `metamodelService.getContainmentTree(pluginName)` and `modelService.getModelTree(pluginName)`. When both resolve, the results are merged into `treeData` and `render()` is called, which walks the structure recursively and generates DOM elements for each node. Expand and collapse buttons manipulate `expandedNodes` and trigger incremental DOM updates without re-fetching from the backend.

PropertiesPanel and MainEditor give the user detailed, editable information about the selected node. PropertiesPanel focuses on intrinsic properties: attributes, references, and operations. MainEditor provides navigational context through breadcrumbs and a local children view.

PropertiesPanel tracks currentNode and currentTab. When showNode(node) is called, the data needed for the active tab is fetched. For the Attributes tab, ModelService.getObjectAttributes is called for current values and MetamodelService.getClassifierDetails for types and multiplicity; type-appropriate input controls are then rendered (text inputs for EString, number inputs for EInt, dropdown selects for enumeration attributes). The References tab distinguishes containment references, shown as child instance lists, from non-containment references, shown with a target selector widget. The Operations tab renders the available operation signatures as reported by the Generic API reflection layer; behavioral execution is deferred to future work, but exposing the signatures keeps the interface a complete reflection of the Ecore model. Attribute edits construct a command object and route it through CommandManager.

MainEditor constructs a breadcrumb trail from the navigation history held by ModelerApp, allowing the user to navigate upward through ancestors. Below the breadcrumbs, a local children view is rendered: for classifier nodes, children are the classifiers reachable via containment references; for instance nodes, children are the immediate child instances from the instance tree. Both panels receive their data from MetamodelService and ModelService, with no direct coupling to other UI components.

CreationPalette, ContextMenu, and CreateInstanceDialog cooperate to drive the instance creation workflow. CreationPalette identifies what can be created; ContextMenu provides the entry point on right-click; and CreateInstanceDialog collects and validates input before dispatching a creation command.

CreationPalette is a floating element that updates on each selection change. When updateForNode(node) is called, MetamodelService.getValidChildTypes is queried for the set of classifiers that may legally appear as containment children of the selected element, and each valid type is rendered as a clickable entry. Clicking an entry opens the dialog.

ContextMenu activates on right-click, triggered by app.onNodeRightClick(node, event). It positions itself at the cursor and renders context-sensitive actions depending on node type: for classifiers, “New Instance” and “Show Properties”; for instances, “New Child”, “Delete”, “Show Properties”, and “Validate”. Each action delegates to the appropriate ModelerApp method.

CreateInstanceDialog is a modal form. When show(parentNode, classifierType)

is called, classifier details are fetched from `MetamodelService` and typed form fields are generated dynamically. For a child instance, the containment reference name is shown as a read-only field. On submission, `ValidationService.validateInstanceCreation` is called; if validation fails, inline error messages appear and the form is held open. If valid, `app.onCreateInstance(...)` is called, which constructs a creation command and passes it to `CommandManager.executeCommand`.

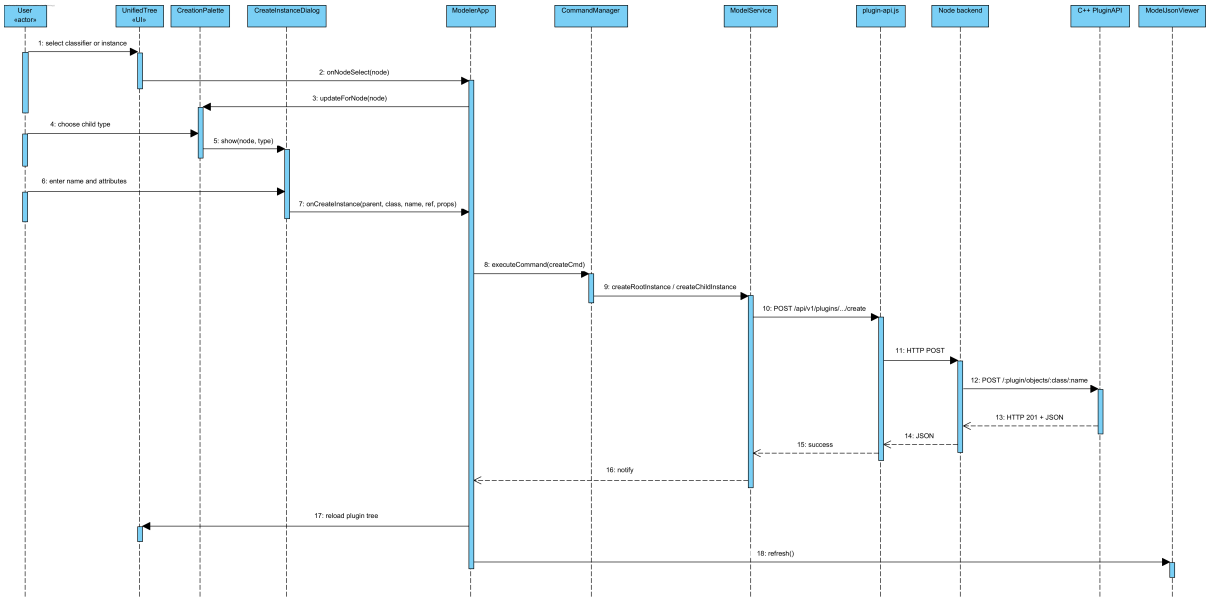


Figure 4: Instance creation from the palette to the C++ PluginAPI.

`ModelJsonViewer` occupies the central panel and provides a live, togglable textual view of the model state in both canonical JSON and XMI format. It serves as both a debugging aid and an export tool. When `refresh()` is called, triggered by any selection change or model mutation, `modelSerializer.buildDocument` is called and the result is rendered with syntax highlighting. In XMI mode, the JSON is passed through `xmiSerializer.toXmi` before rendering. Three toolbar actions are exposed: mode toggle, copy-to-clipboard, and file download.

6.3.6 Data Persistence and Export

`modelSerializer` builds a canonical JSON document capturing both metamodel structure and current instance state. It works in two phases: first the `metamodel` section is assembled by calling `MetamodelService.getClassifiers` and `getClassifierDetails` for each classifier; then the `model` section is built by calling `ModelService.getModelTree` and `getObjectAttributes` for each instance. The combined schema is stable across plugins and suitable for archiving, diffing, or feeding into external tooling.

`xmiSerializer` converts the canonical JSON from `modelSerializer` into an XMI 2.0 document. Each object is represented as an XML element whose tag is the classifier

name qualified with the metamodel namespace prefix. Attributes are serialized as XML attributes on that element. Containment references produce nested child elements; non-containment references are written as `href` attributes. `xsi:type` attributes are added where needed, and the result is a valid, formatted XMI 2.0 document.

7 Results

This chapter presents the empirical results gathered during the prototype evaluation. The behavior of the system is examined through its primary usage flows, after which the outcomes of the four validation tests are discussed alongside visual evidence of the interface in operation.

7.1 System Dynamics and User Interactions

The coordination between the browser, the Node.js proxy, and the C++ reflective engine is perhaps most evident when tracing the primary usage flows. These interactions demonstrate how the three-tier architecture maintains state consistency across different runtime environments.

7.1.1 Model Initialization and Plugin Handshake

When a session is initiated, a query is first sent by the `ModelerApp` to the Node backend to retrieve available plugins. Once a selection is made, parallel requests are issued by the `UnifiedTree` for both the metamodel hierarchy and the current instance tree. It was observed that this parallel execution is necessary for maintaining UI responsiveness, especially as metamodel depth increases.

Generating a model instance emerged as the most intricate flow in the system. The process begins when a user selects a classifier from the `CreationPalette`, which triggers a check by the `ValidationService` for mandatory attributes and cardinality constraints within the browser. A `CreateInstanceCommand` is then dispatched, resulting in an asynchronous POST request. This request is normalized by the Node.js backend before being passed to the C++ `PluginAPI`. Following the split-phase lock insertion by the native server, a success record is returned, and a recursive refresh of the visual and textual viewers is finally triggered.

7.2 Experimental Evaluation

The evaluation was structured as four distinct tests, each designed to verify a core claim of the prototype regarding metamodel neutrality, metadata integrity, and system resilience.

7.2.1 Test 1: Plugin-Agnostic Interface Behavior

The project rests on the claim that the interface can function with any loaded MDE4CPP plugin without requiring modifications to the frontend source code. Validation was performed by testing the system against two structurally distinct metamodels: a standard

library model and a complex UML profile. In both instances, the classifiers were successfully reflected [1], and the UI components automatically adapted their controls to the specific metamodel without manual intervention.

7.2.2 Test 2: Metadata Consistency Under Load

Observation was primarily focused on whether the logical associations between parent and child elements remained consistent during high-frequency creation and deletion operations. The string-based instance identifiers used in the interaction layer appeared to provide a stable mapping, which kept the visual tree synchronized with the native object state even under intensive reflective activity.

7.2.3 Test 3: Interoperability and XMI Schema Compliance

For a model-driven tool, interoperability is a significant factor. XMI output from the web modeler was imported into Eclipse EMF [2] and Papyrus to verify namespace correctness and attribute mapping. The `xmiSerializer.js` module was found to produce XMI 2.0 compliant documents [13] that were successfully reopened and edited in desktop modeling environments.

7.2.4 Test 4: System Resilience and Fallback Analysis

The dual-mode fallback logic was evaluated by terminating the C++ `pluginAPI.exe` process during an active session. It was observed that the system transitioned gracefully to a read-only state, serving technical metadata from the browser cache and instance data from the in-memory store. This transition prevented browser hangs and allowed the user to continue browsing the existing model state.

The three-tier architecture appears to successfully connect native performance to web accessibility. The prototype demonstrates a level of metamodel neutrality and structural stability that provides a useful starting point for future browser-based tooling.

7.3 Syntactic and Structural Validation

Visual validation involved a direct comparison between the runtime model in the browser and its serialized XMI representation. The sample below illustrates a correctly structured XMI export where the namespace prefix and containment hierarchy match the in-memory state.

```xml
<?xml version="1.0" encoding="UTF-8"?>
<library:Library xmi:version="2.0"
    xmlns:xmi="http://www.omg.org/XMI"
    xmlns:library="http://www.example.org/library"
    name="University Library">
  <books name="Design Patterns" author="Gamma et al." year="1994"/
    >
  <books name="Model-Driven Engineering" author="Schmidt" year="
    2006"/>
</library:Library>
```

Listing 2: Sample Exported XMI showing deep nesting and attribute mapping.

7.4 Summary of Achievements

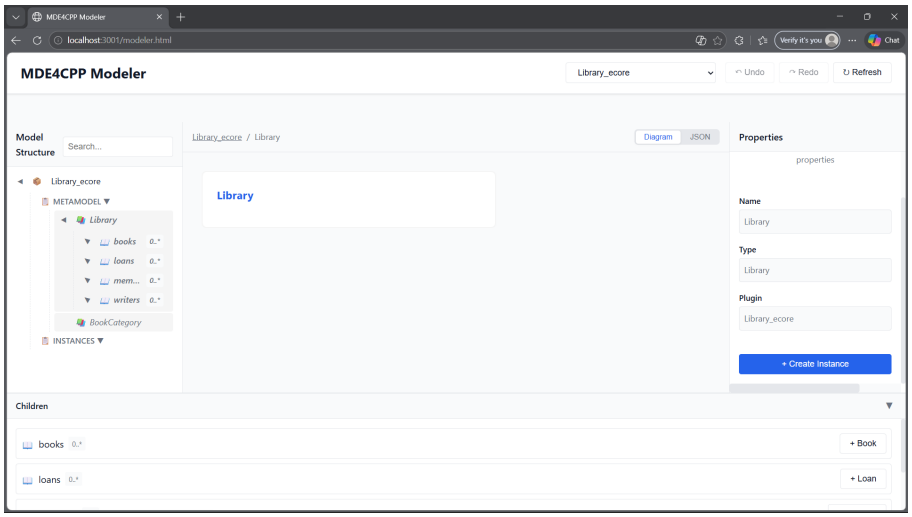
Table 8 maps each main contribution to the problem it addressed and the evidence of its successful implementation.

Table 8: Contributions: problem, solution, and evidence.

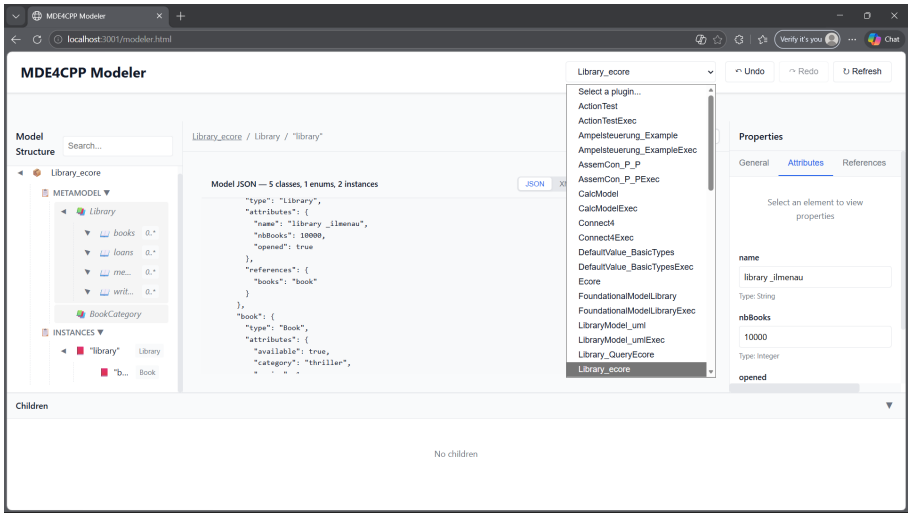
Area	Problem	Solution	Evidence
Build	PluginAPI build failure on machines without a system-wide Boost installation [7].	Introduction of the `deliverBoostLibrary` Gradle task with three-way resolution.	Successful Gradle build from scratch on a clean environment.
Prototype Adaptation	Complexity of direct reflective manipulation and lack of concurrency during the discovery phase.	Implementation of lightweight instance tracking and mutex-based synchronization.	Stable handling of concurrent web requests; all new endpoints verified end-to-end.
Web Modeler	Absence of a browser-based interface for the MDE4CPP runtime.	Development of a three-tier web application integrating the browser, Node.js [8], and C++.	Functioning modeler with tree navigation, CRUD operations, and XMI export [13].

7.5 Visual Evidence

This section centralizes the primary interface states observed during the validation process. These figures document the successful reflection of metamodels and the dynamic generation of UI controls.



(a) Metamodel tree navigation with reflective property updates.



(b) The plugin selector dropdown showing all dynamically discovered plugins.

Figure 5: Primary interface states (Part I): Metamodel reflection and plugin discovery.

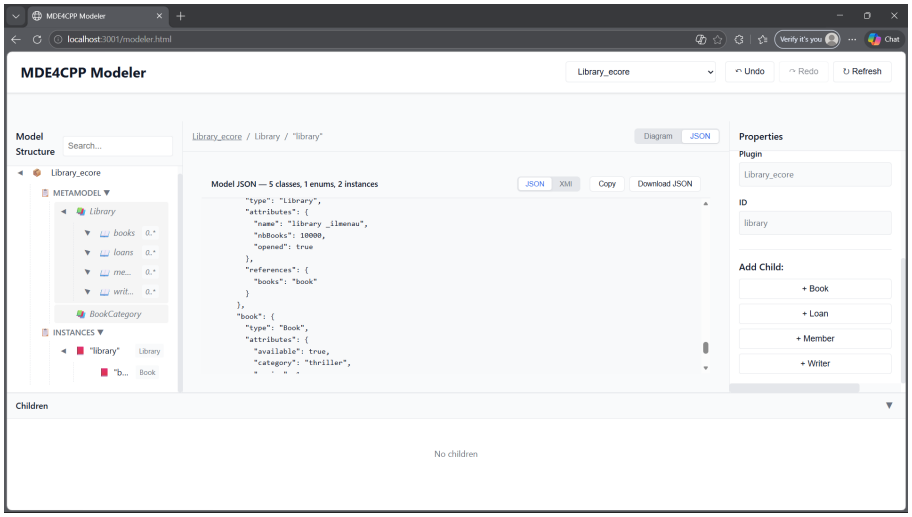


Figure 5: Primary interface states (Part II): The JSON and XMI viewer panel reflecting live model state.

8 Conclusion

The objective of this project was the development of a functioning web-based modeler designed to interact with the MDE4CPP runtime, without being restricted to any single specific model. That objective was achieved. This final chapter summarizes the results, reflects on the limitations of the current prototype, and presents a roadmap for the transition of this tool beyond its current stage.

8.1 Achievements

Three primary contributions resulted from this work. First, a reproducible build foundation was established through the introduction of the `deliverBoostLibrary` Gradle task, which handles the resolution and delivery of Boost 1.82.0 headers [7]. This removed a known friction point for developers. Second, a plugin-agnostic web modeler was developed, demonstrating that a unified modeling interface is feasible. By building on the reflective contract of the C++ `GenericApi`, the frontend was designed to adapt to any MDE4CPP plugin, from simple Library models to complex UML distributions [1], without requiring modifications to the modeler source code.

8.2 Academic and Architectural Implications

The three-tier proxy architecture (Browser → Node.js → C++) presents a pattern that might be useful to the broader MDE community. It was observed that typical concerns such as native runtime exposure and error propagation can be addressed within a lightweight mediation layer. The reflective interface of the `GenericApi` proved more resilient than initially expected, and the patterns developed for managing `eGet` and `eSet` operations across an HTTP boundary appear to generalize well to other modeling contexts.

8.3 Limitations and Critical Reflection

The prototype as it stands has certain limitations. The most significant is the absence of persistent storage, as all model instances are held only in the memory of the `PluginAPI`. While XMI export [13] provides a manual method for saving state, the risk of data loss remains without an automated persistence layer. In addition, the current instance tracking relies on string-based identifiers rather than native `EObject` references. While this was adopted as a practical measure for the prototype phase, a transition to native references would likely yield higher efficiency and better alignment with the core design of MDE4CPP.

8.4 Future work

The evolution of the MDE4CPP web interaction layer is expected to involve several stages. In the short term, efforts might be directed toward adding persistent storage via a document-oriented database and implementing secure authentication. The expansion of functional support to include `invokeOperation` would allow behavioral features to be triggered directly from the browser. In the medium term, a transition to a reactive frontend framework with a graphical canvas could be considered to enhance the user experience. Ultimately, the achievement of collaborative modeling will require the implementation of sophisticated session management and conflict resolution strategies.

9 Glossary

Metamodel	A formal description of a modeling language using classifiers, attributes, references, and operations. Ecore and UML are typical examples.
Model	A concrete artifact conforming to a metamodel, composed of object instances and links that satisfy the constraints defined in the metamodel.
PluginFramework	The MDE4CPP runtime component responsible for discovering, loading, and managing model plugins (Ecore, UML, fUML, PSCS); it provides the reflection API utilized by the PluginAPI.
PluginAPI	A standalone C++ executable that exposes model management and metamodel reflection over HTTP using the Crow framework [9].
GenericApi	The central class in the PluginAPI that manages m_objects, m_plugins, and the synchronization mutex while registering all HTTP routes.
Node.js Backend	The Express-based HTTP server [10] located in interface/backend that exposes /api/v1 routes and forwards requests to the C++ PluginAPI.
Modeler Frontend	The browser-based single-page application located in interface/frontend.
ModelerApp	The central controller module that manages global selection state and coordinates events across UI components.
UnifiedTree	The UI component in the left panel that displays the combined metamodel and instance tree for the currently selected plugin.
CommandManager	The module that implements the command pattern to provide undo and redo functionality using dual stacks [12].
Data Access Layer	A collection of modules including plugin-api.js, MetamodelService, ModelService, and ValidationService that handle all communication with the backend.
Controller Layer	The architectural layer comprising ModelerApp and Command Manager, responsible for global state management and command sequencing.
Presentation Layer	The layer containing all UI component modules that handle rendering and user interaction.

deliverBoostLibrary	The Gradle task that ensures the availability of Boost 1.82.0 headers before the PluginAPI is compiled.
Crow	A lightweight C++ HTTP micro-framework used as the embedded server within the PluginAPI [9].
XMI	An XML-based exchange format for model artifacts [13]; it is generated by xmiSerializer from the canonical JSON output produced by modelSerializer.

References

[1] Ralph Maschotta, Alexander Wichmann, and Armin Zimmermann. "MDE4CPP: An Approach for Generating C++ Code from EMF and UML Models". In: *2018 IEEE/ACM 21st International Conference on Model Driven Engineering Languages and Systems (MODELS)*. IEEE. 2018, pp. 55–65.

[2] Dave Steinberg et al. *EMF: Eclipse Modeling Framework*. Addison-Wesley Professional, 2008.

[3] Object Management Group. *Unified Modeling Language (UML) Specification, Version 2.5.1*. Available at: `https://www.omg.org/spec/UML/2.5.1/`. 2017.

[4] Object Management Group. *Semantics of a Foundational Subset for Executable UML Models (fUML), Version 1.3*. Available at: `https://www.omg.org/spec/FUML/1.3/`. 2017.

[5] Object Management Group. *Object Constraint Language (OCL), Version 2.4*. Available at: `https://www.omg.org/spec/OCL/2.4/`. 2014.

[6] Object Management Group. *Precise Semantics of UML Composite Structures (PSCS), Version 1.2*. Available at: `https://www.omg.org/spec/PSCS/1.2/`. 2018.

[7] Boost Community. *The Boost C++ Libraries*. `https://www.boost.org/`. 2024.

[8] OpenJS Foundation. *Node.js: A JavaScript Runtime Built on Chrome's V8 JavaScript Engine*. `https://nodejs.org/`. 2024.

[9] Crow Team. *Crow: A Fast and Easy to Use Microframework for the C++ Programming Language*. `https://crowcpp.org/`. 2024.

[10] OpenJS Foundation. *Express: Fast, Unopinionated, Minimalist Web Framework for Node.js*. `https://expressjs.com/`. 2024.

[11] Douglas C Schmidt. "Model-driven engineering". In: *Computer* 39.2 (2006), pp. 25–31.

[12] Erich Gamma et al. *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley Professional, 1994.

[13] Object Management Group. *XML Metadata Interchange (XMI) Specification, Version 2.5.1*. Available at: `https://www.omg.org/spec/XMI/2.5.1/`. 2015.